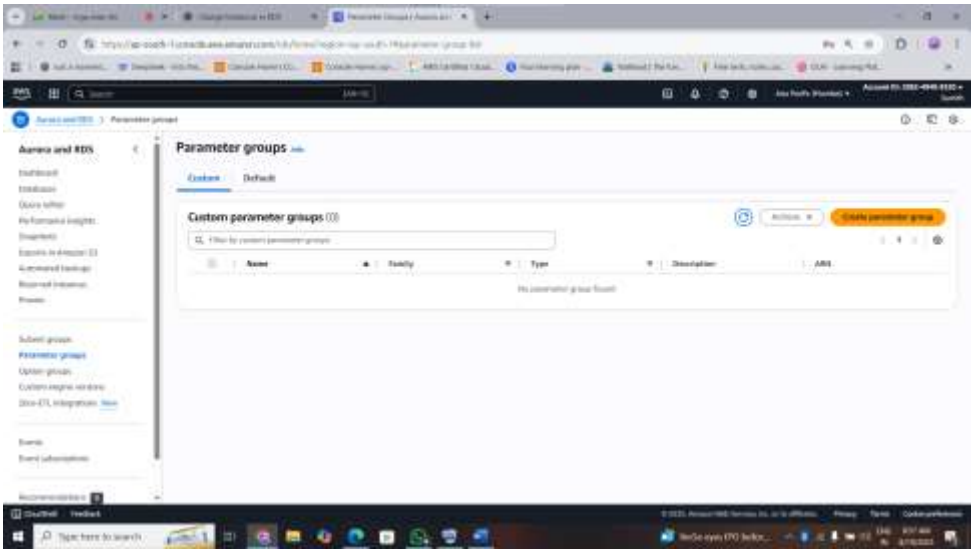
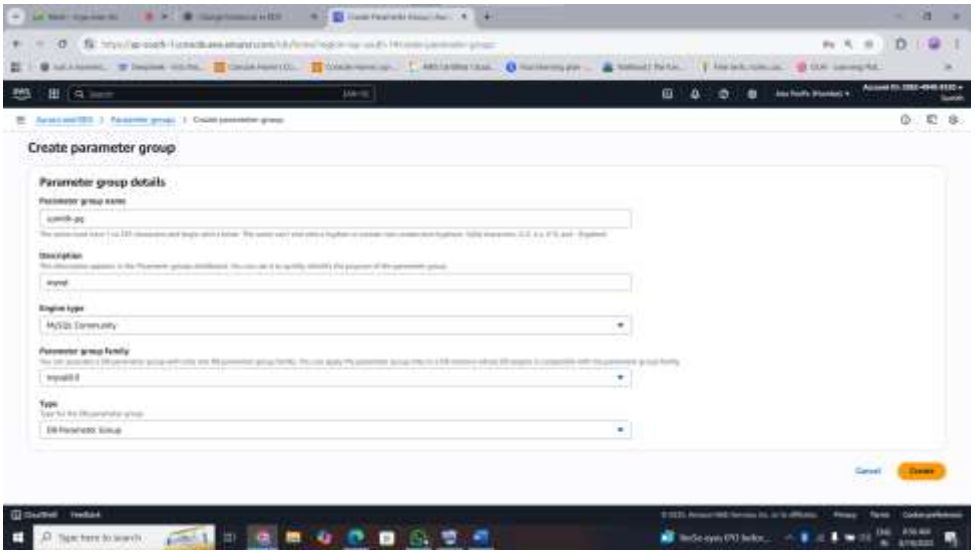
Task: Create & Modify Custom DB Parameter Group in RDS

1. Create a Custom Parameter Group

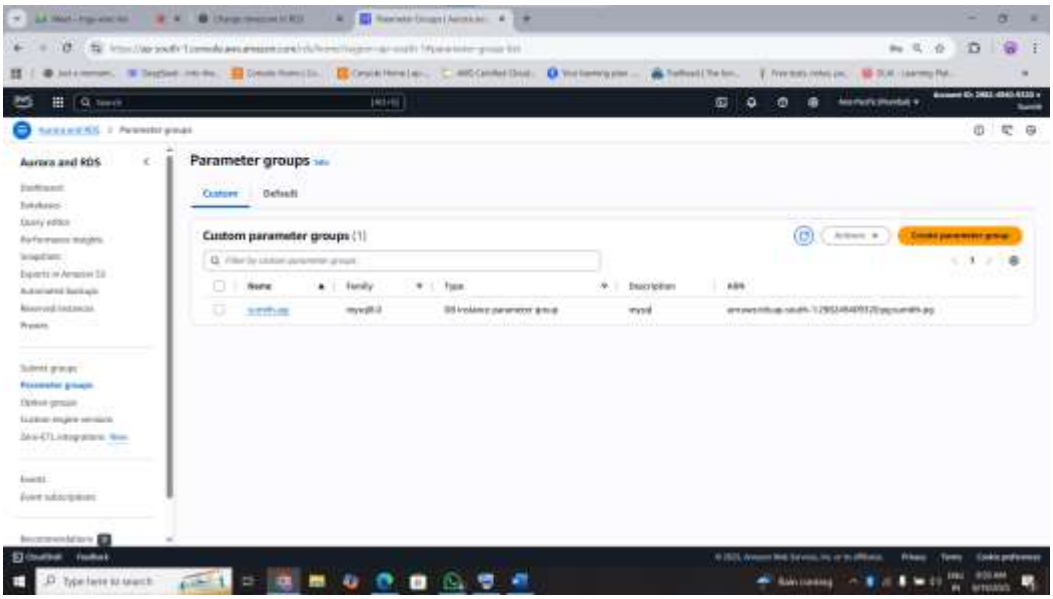
1. Go to **AWS Management Console → RDS**.
2. In the left pane, select **Parameter groups**.
3. Click **Create parameter group**:
   - Parameter group family: choose the right family (e.g., mysql8.0 if your RDS is MySQL 8.0).
   - Type: DB Parameter Group.
   - Name: e.g., Sumith-pg.
   - Description: "mysql".

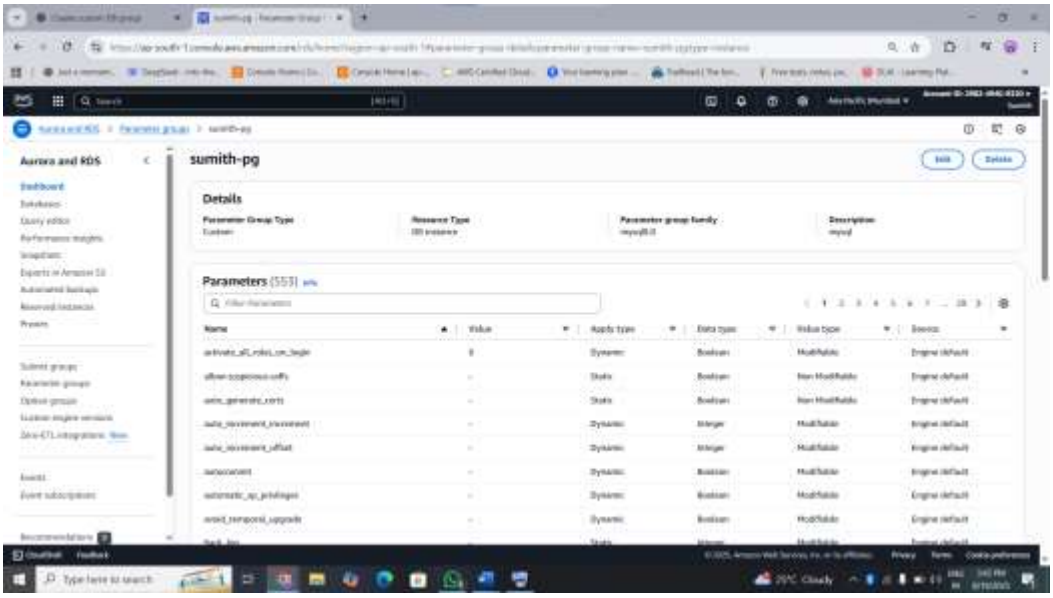




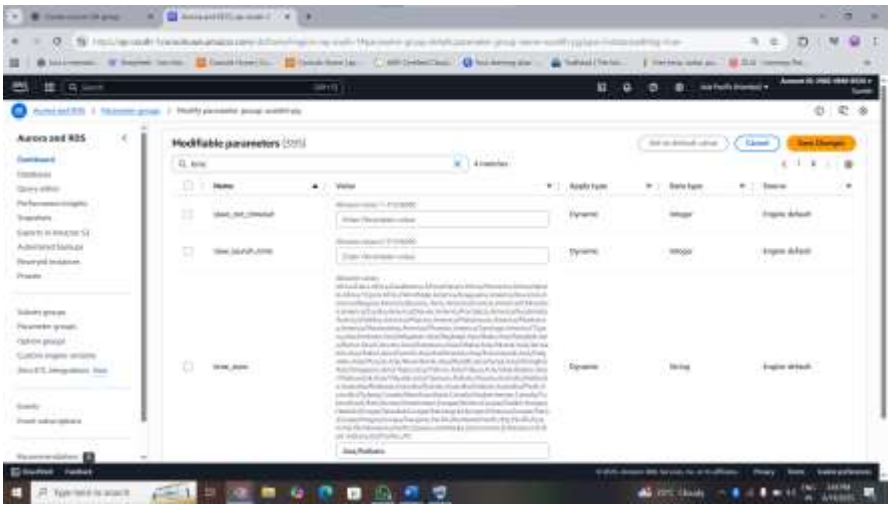
2. Modify Parameters

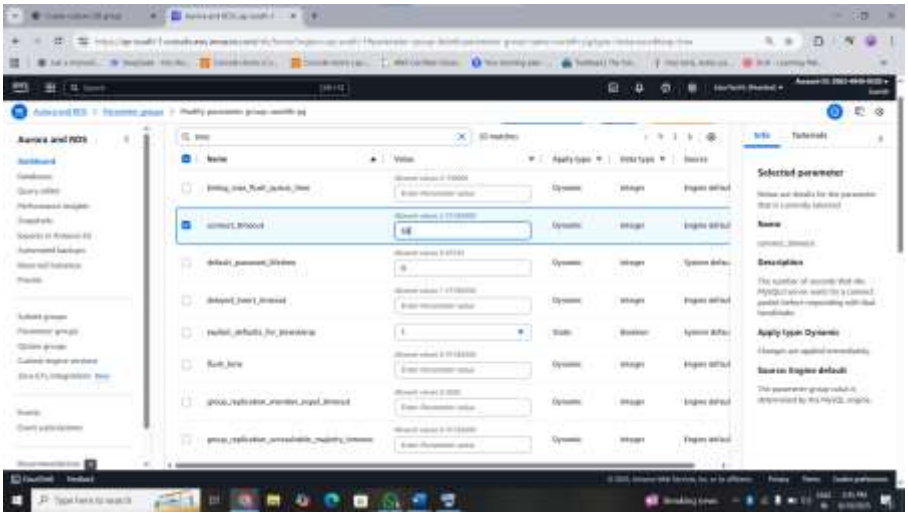
1. Select your newly created parameter group.
2. Click **Edit parameters**.

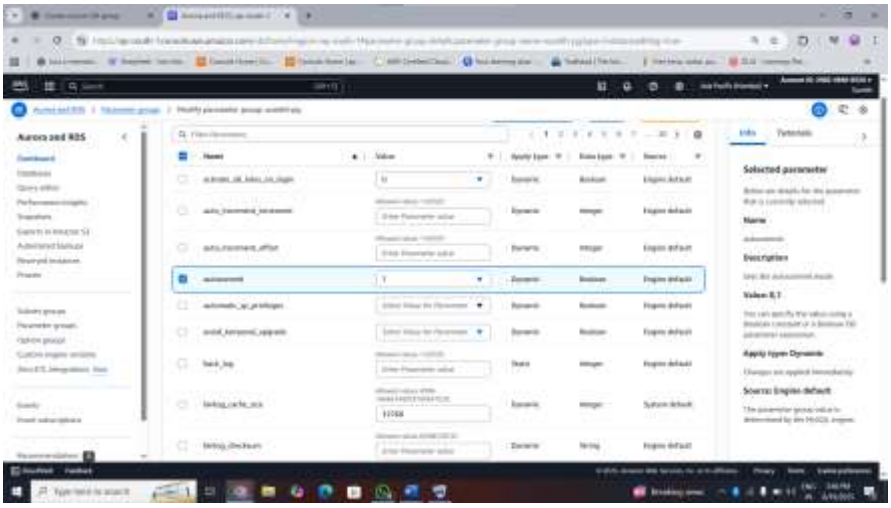




3. Search and set:
    - time_zone = Asia/Kolkata
    - connect_timeout = 60
    - auto commit = 1 (ON) or 0 (OFF)







💡 Notes:

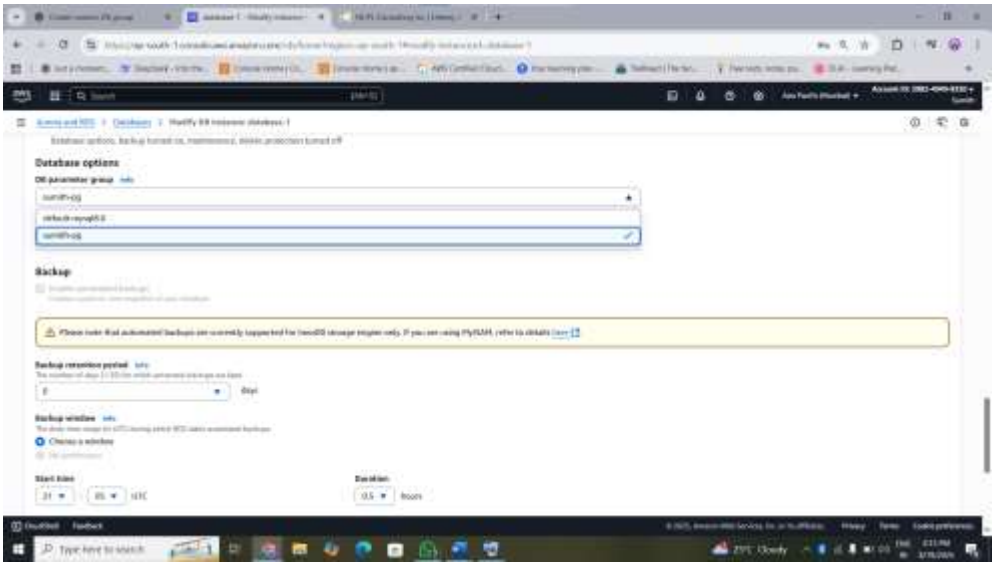
- time_zone applies after **instance reboot**.
- connect_timeout defines how long (in seconds) the server waits for a client connection.
- Auto commit changes the default commit mode for sessions.

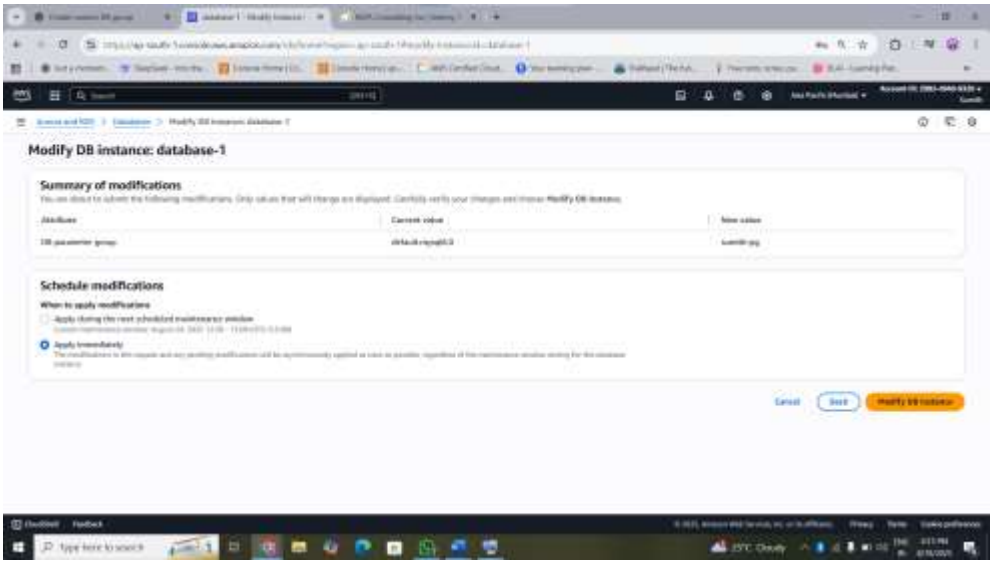
Save the changes.

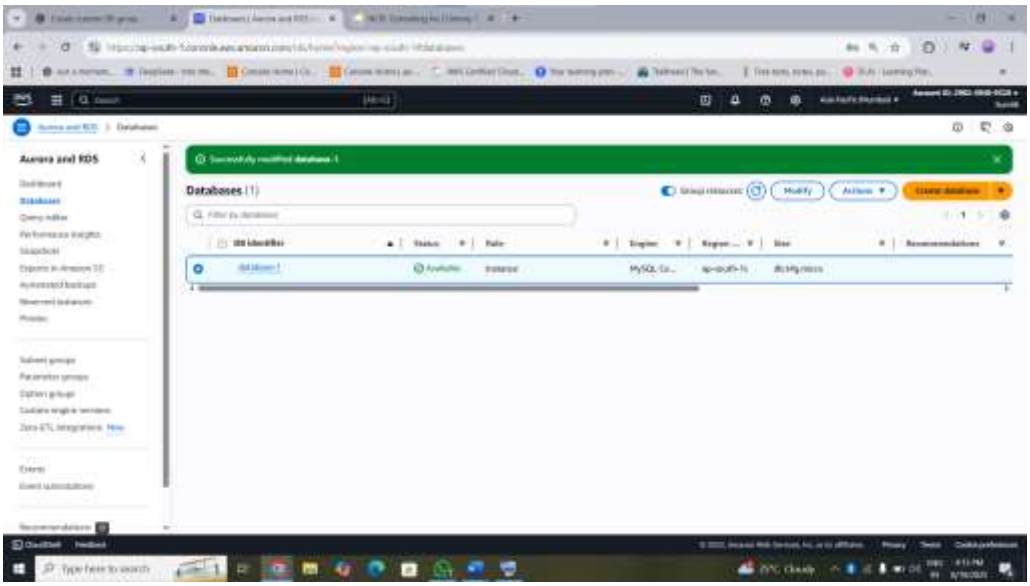
---

3. Apply the Parameter Group

1. Go to **Databases → Your RDS instance**.
2. Choose **Modify**.
3. Under **Database options → DB parameter group**, select custom-mysql-params.
4. Save changes and choose **Apply immediately** (or during next maintenance).
5. For time_zone, reboot the instance once.

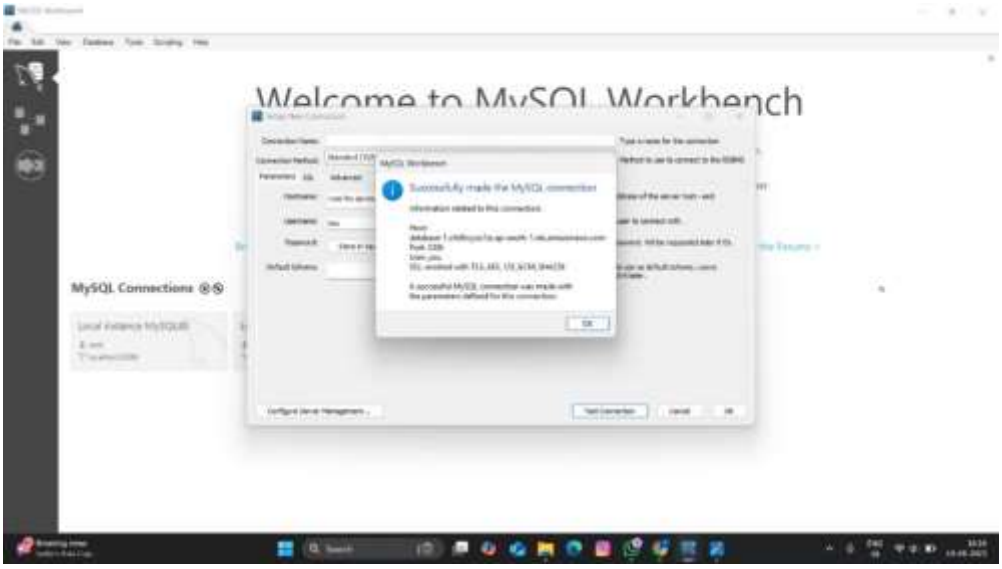

Database options
DB parameter group
sumith-pg
default.mysql8.0
sumith-pg
Backup
Backup retention period
Backup window
Choose a window
Start time
Duration
Hours


Modify DB instance: database-1
Summary of modifications
Attribute
Current value
New value
DB parameter group
default.mysql8.0
sumith-pg
Schedule modifications
When to apply modifications
Apply during the next scheduled maintenance window
Apply immediately
Cancel
Back
Modify DB instance


Aurora and RDS
Successfully modified database-1
Databases (1)
Modify
Actions
Create database
DB identifier
Status
Role
Engine
Region ...
Size
Recommendations
database-1
Available
Instance
MySQL Co...
ap-south-1c
db.t4g.micro

4. Verification via MySQL

Connect to your RDS instance using MySQL Workbench or CLI:

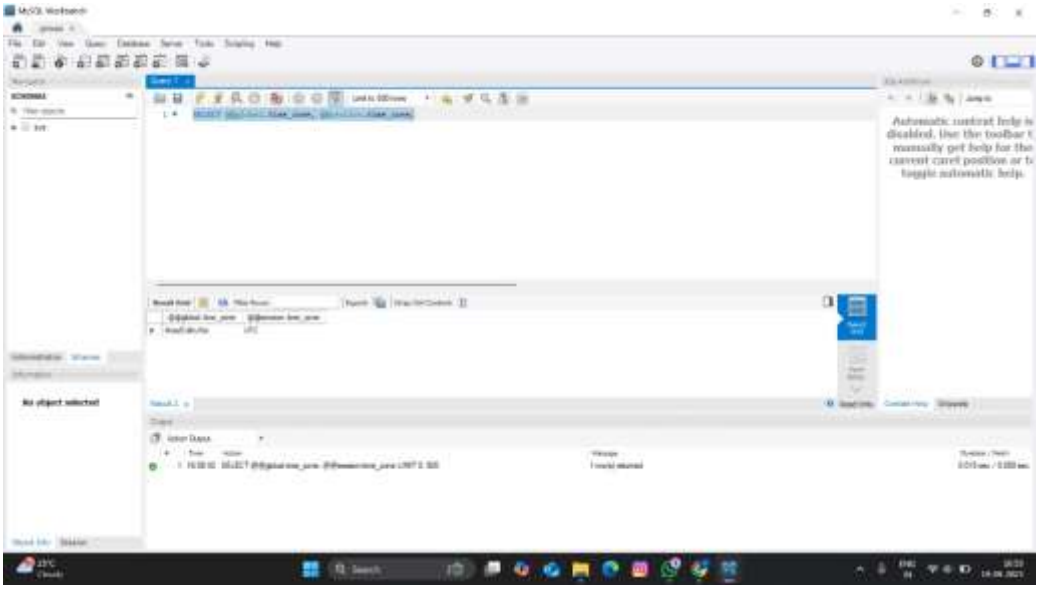


mysql -h <RDS-endpoint> -u <username> -p

Run queries to verify:

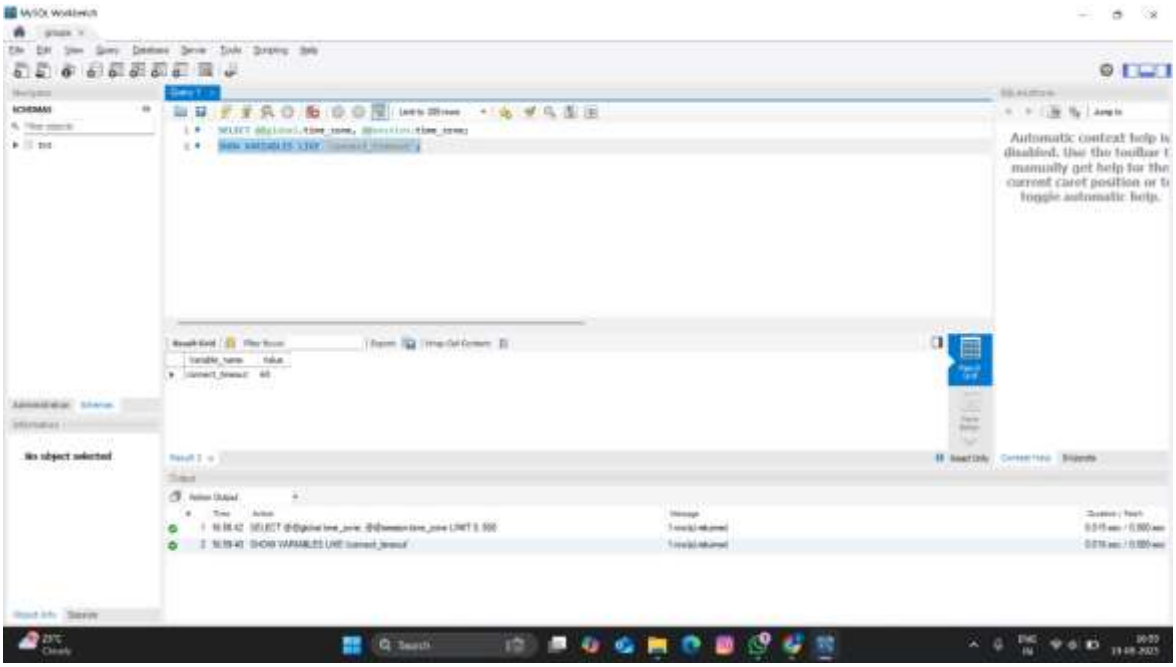
- **Time Zone**

SELECT @@global.time_zone, @@session.time_zone;



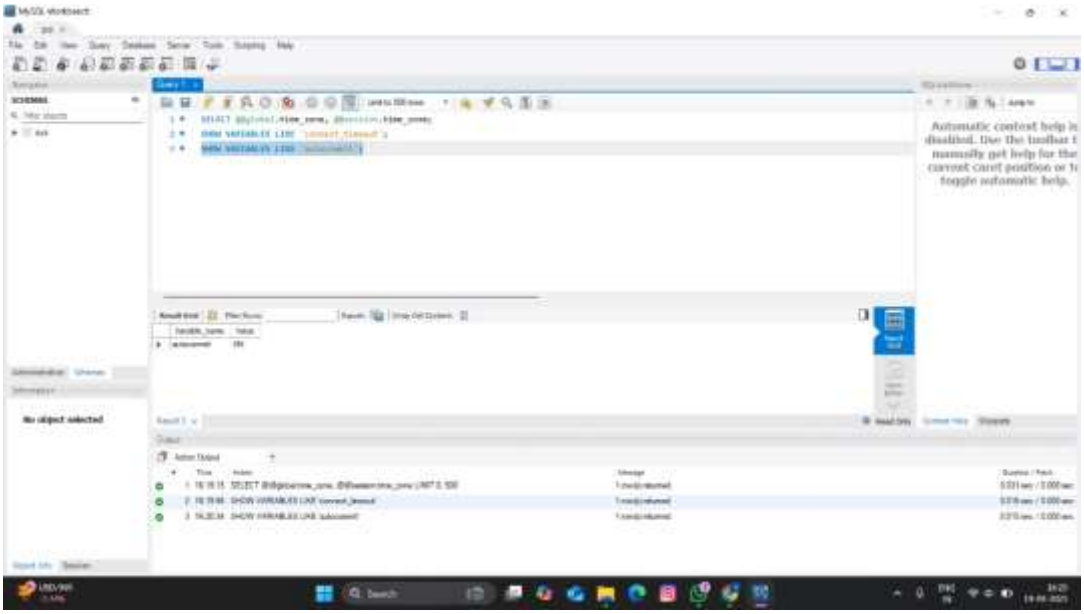
- **Connection Timeout**

SHOW VARIABLES LIKE 'connect_timeout';



- **Autocommit Mode**

SHOW VARIABLES LIKE 'autocommit';

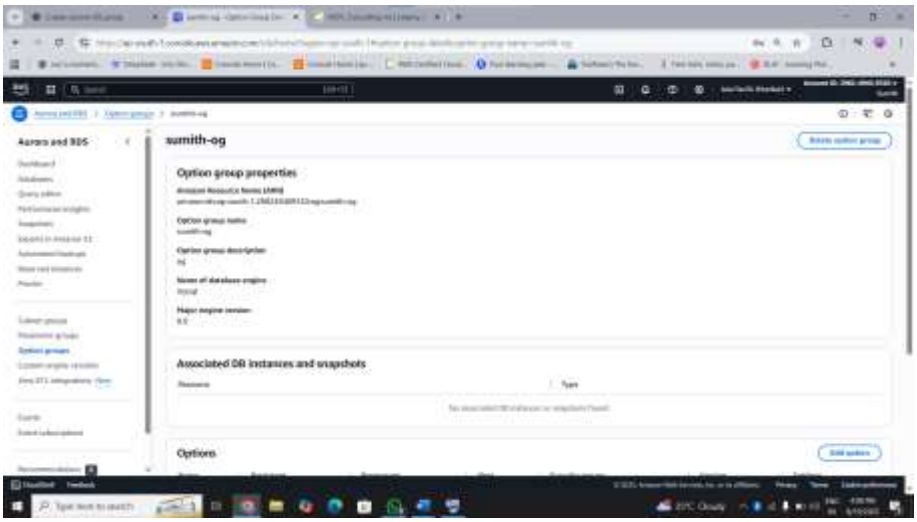


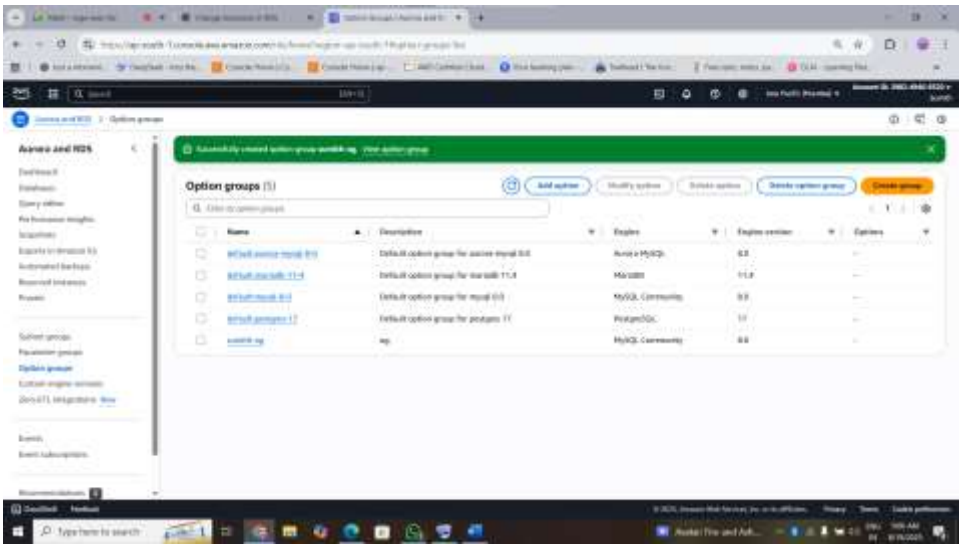
If outputs match your custom settings, parameter group was applied successfully .

Task: Enable MySQL Audit Plugin in RDS

1. Create a New Option Group

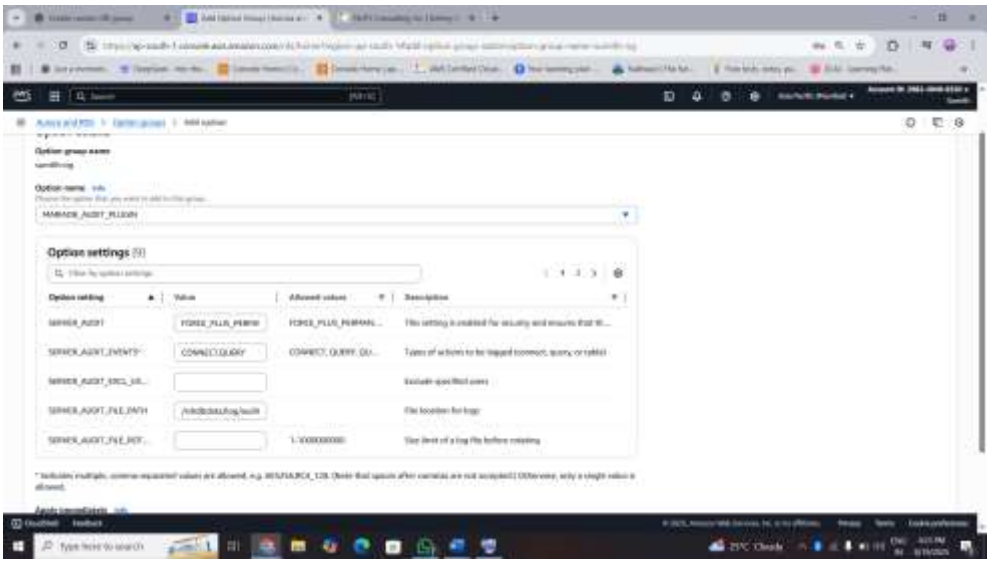
1. Go to **AWS Console → RDS → Option groups**.
2. Click **Create group**:
   - Name: sumith-og
   - Engine: mysql
   - Engine version: match your DB (e.g., MySQL 8.0.35)
   - Description: "Audit plugin for MySQL"
3. Create the group.



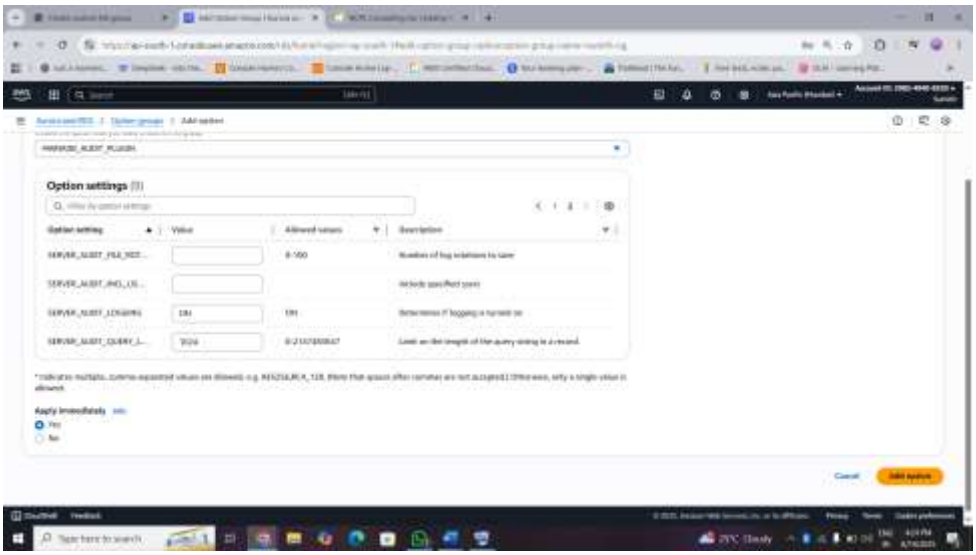


2. Add the Audit Plugin Option

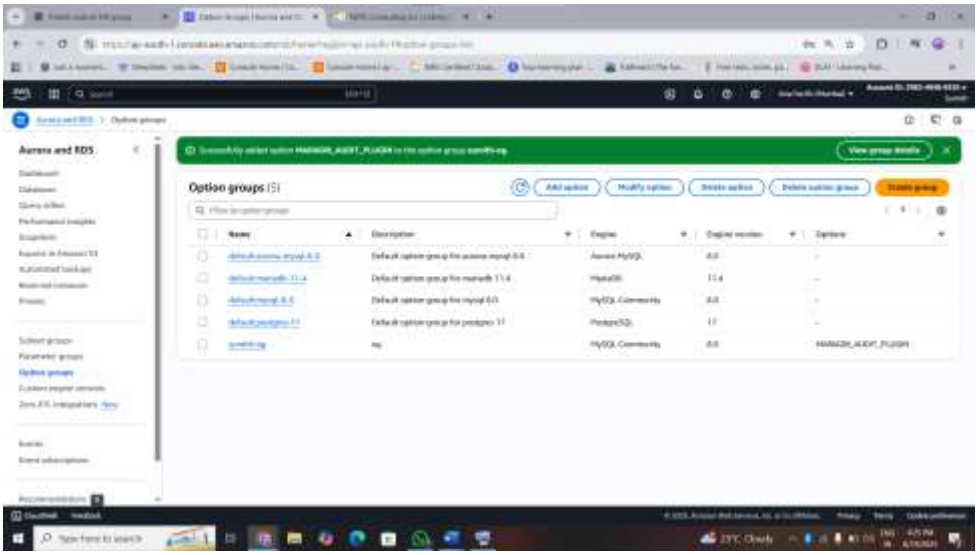
1. Select your option group (sumith-og).
2. Choose **Add option**.
3. Configure:
    - Option name: MARIADB_AUDIT_PLUGIN (for MySQL in RDS).
    - Option settings:
        - SERVER_AUDIT_EVENTS = CONNECT,QUERY



- SERVER_AUDIT_LOGGING = 1



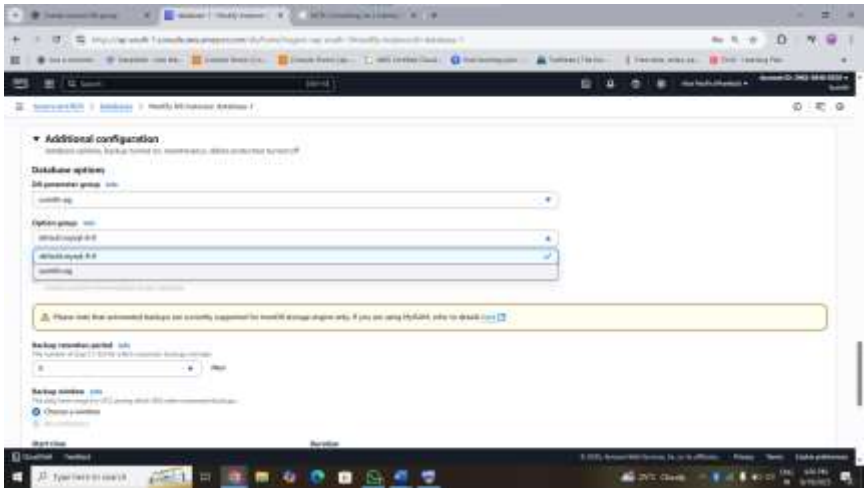
- Apply immediately: **Yes**.
- CloudWatch Logs: enable & pick a log group (so logs go to CloudWatch).

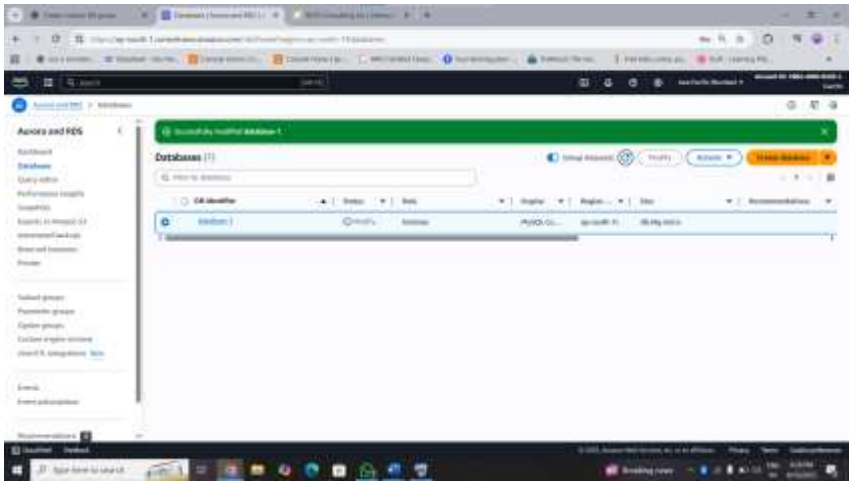


---

3. Apply the Option Group to RDS Instance

1. Go to **Databases → Your RDS instance**.
2. Modify → under **Option group**, select sumith-og.
3. Save and **Apply immediately**.
4. Reboot the DB instance to activate the plugin.





---

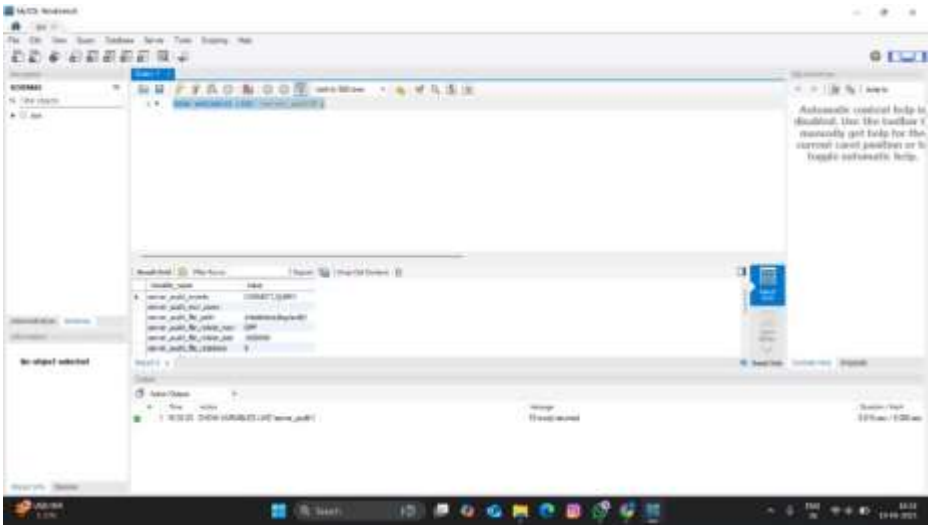
4. Verification

Connect to the DB and confirm plugin is active:

SHOW VARIABLES LIKE 'server_audit%';

Expected:

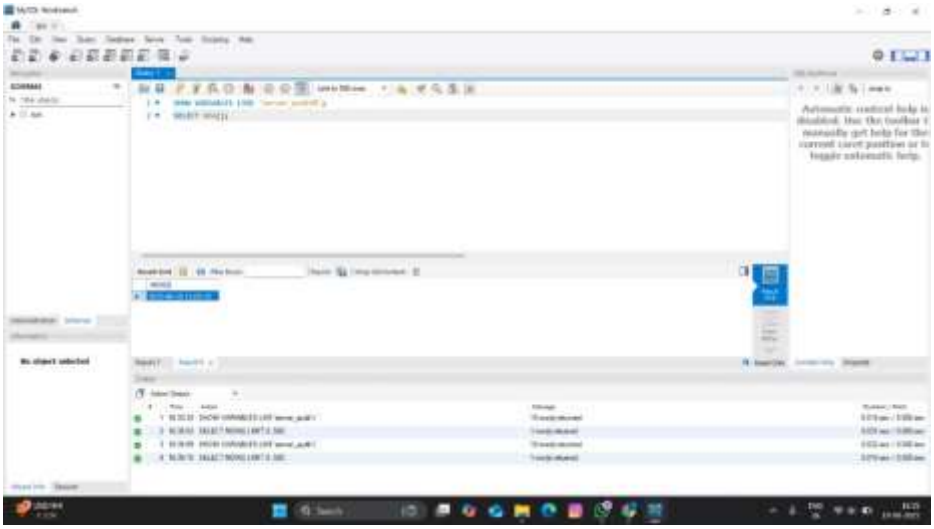
- server_audit_events = CONNECT,QUERY
- server_audit_logging = ON



---

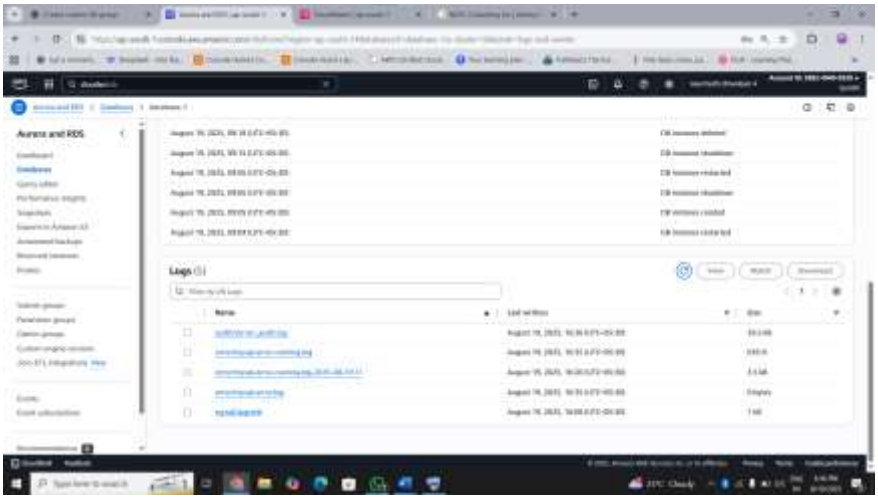
5. Deliverable – Audit Logs

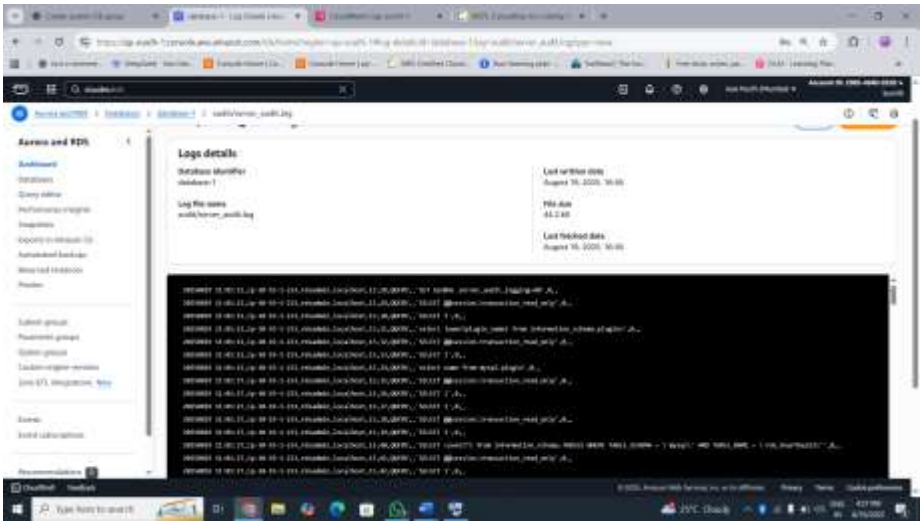
- After enabling, perform test actions:
  - Connect to DB.
  - Run a sample query:
  - SELECT NOW();
- Then check logs:



Option 1 – RDS Console:

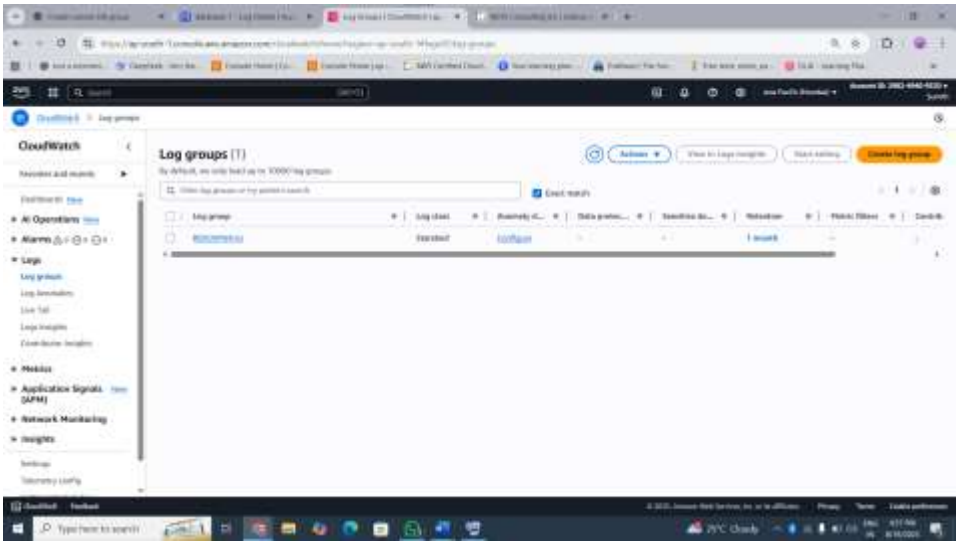
- Go to **RDS → Databases → Instance → Logs & events → Log files**.
- Open audit.log.

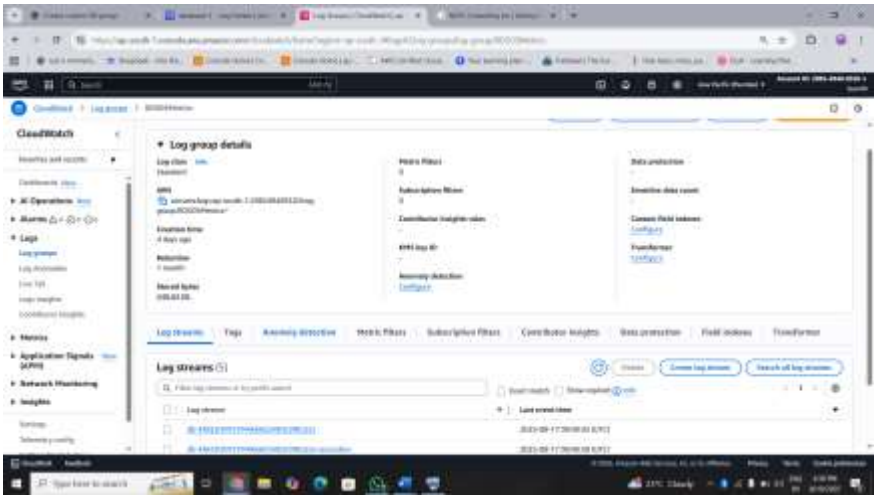




Option 2 – CloudWatch Logs:

- Open **CloudWatch → Log groups → RDSOSMetrics or your custom audit log group**.
- You'll see entries like:
- [Server_audit] Connect; user=root; host=...;
- [Server_audit] Query; user=root; SELECT NOW();





Task: Subnet Group Setup for High Availability

1. Plan Subnets

- You'll need at least **2 public subnets** across **different Availability Zones (AZs)** within the same VPC.
- Example in **us-east-1**:
  - subnet-1 in us-east-1a
  - subnet-2 in us-east-1b

Each subnet must have:

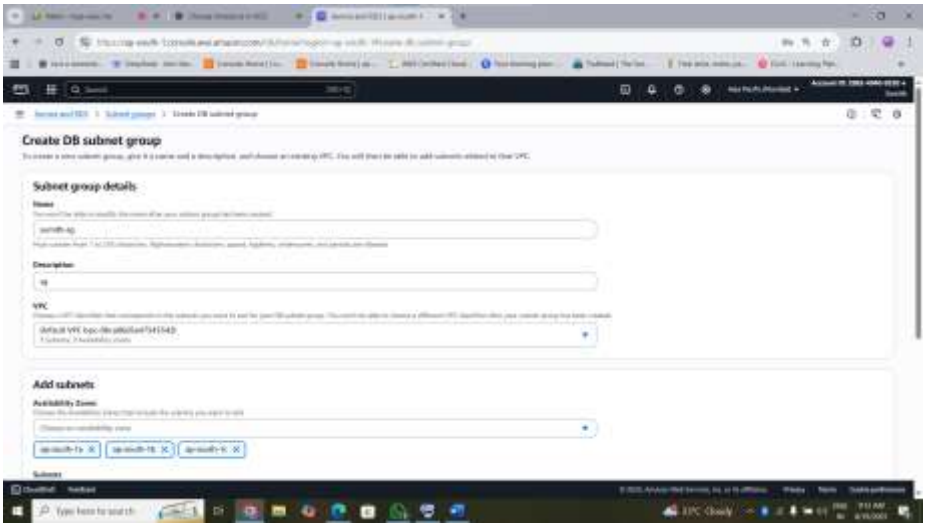
- **Public route table** (with route to Internet Gateway).
- Proper **security groups** and **NACLs** allowing MySQL port (3306) access (from trusted sources only).

---

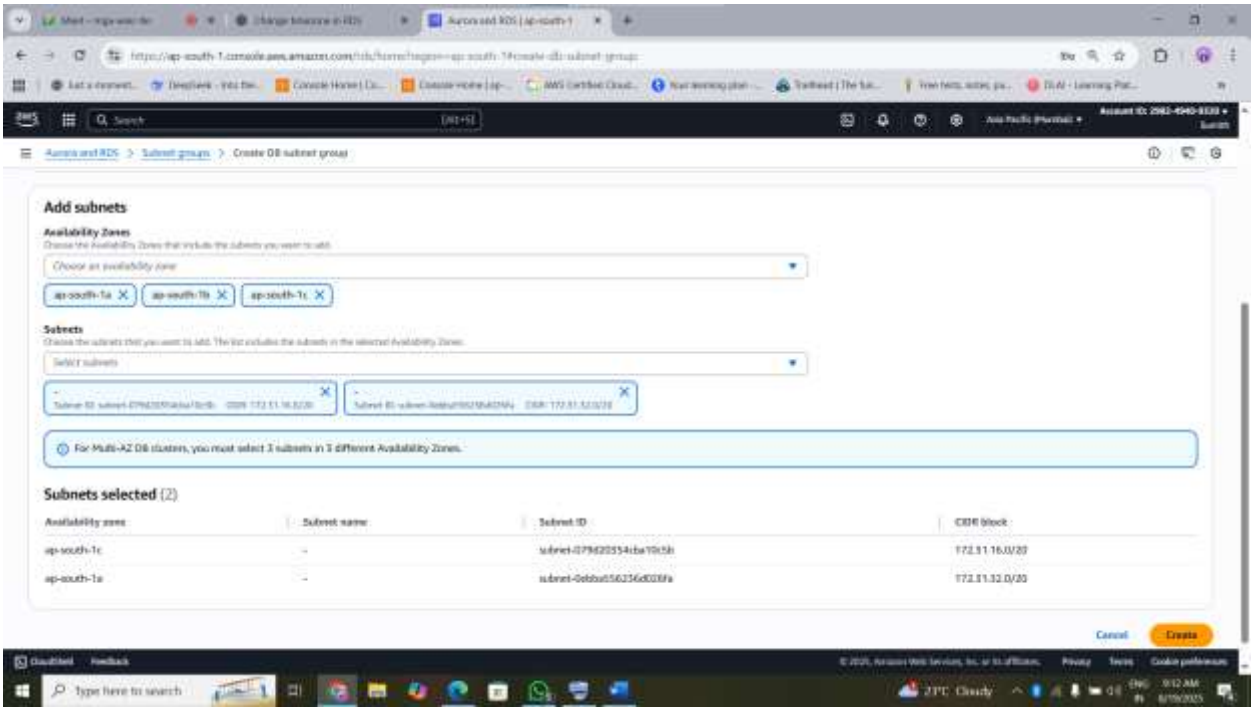
2. Create a Custom Subnet Group

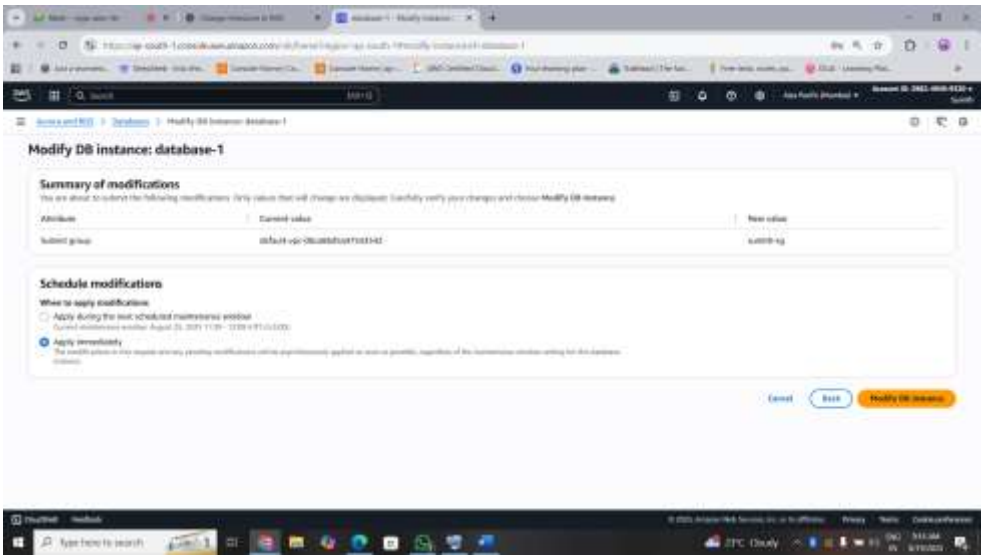
From **AWS Console**:

1. Go to **RDS → Subnet groups**.
2. Click **Create DB subnet group**:
   - Name: ha-subnet-group
   - VPC: choose your VPC
   - Add subnets:
     - Select us-east-1a → public subnet
     - Select us-east-1b → public subnet


Create DB subnet group
Subnet group details
Add subnets
Availability Zones
ap-south-1a
ap-south-1b
ap-south-1c

- Save.


Aurora and RDS > Subnet groups > Create DB subnet group
Add subnets
Availability Zones
Choose an availability zone
ap-south-1a
ap-south-1b
ap-south-1c
Subnets
Select subnets
For Multi-AZ DB clusters, you must select 3 subnets in 3 different Availability Zones.
Subnets selected (2)
Availability zone
Subnet name
Subnet ID
CIDR block
ap-south-1c
-
subnet-079d205554cba10c5b
172.31.16.0/20
ap-south-1a
-
subnet-0ebba556236d03bfa
172.31.32.0/20
Cancel
Create



💡 Rule: A subnet group must include **at least two subnets in different AZs** for Multi-AZ support.

---

3. Associate Subnet Group with RDS Instance

1. Launch a new RDS instance or modify an existing one.
2. Under **Connectivity**, choose:
   - **VPC** → your VPC
   - **Subnet group** → ha-subnet-group
   - **Public access** → Yes (since subnets are public).
3. Enable **Multi-AZ deployment** (this creates a standby DB in the other AZ).

---

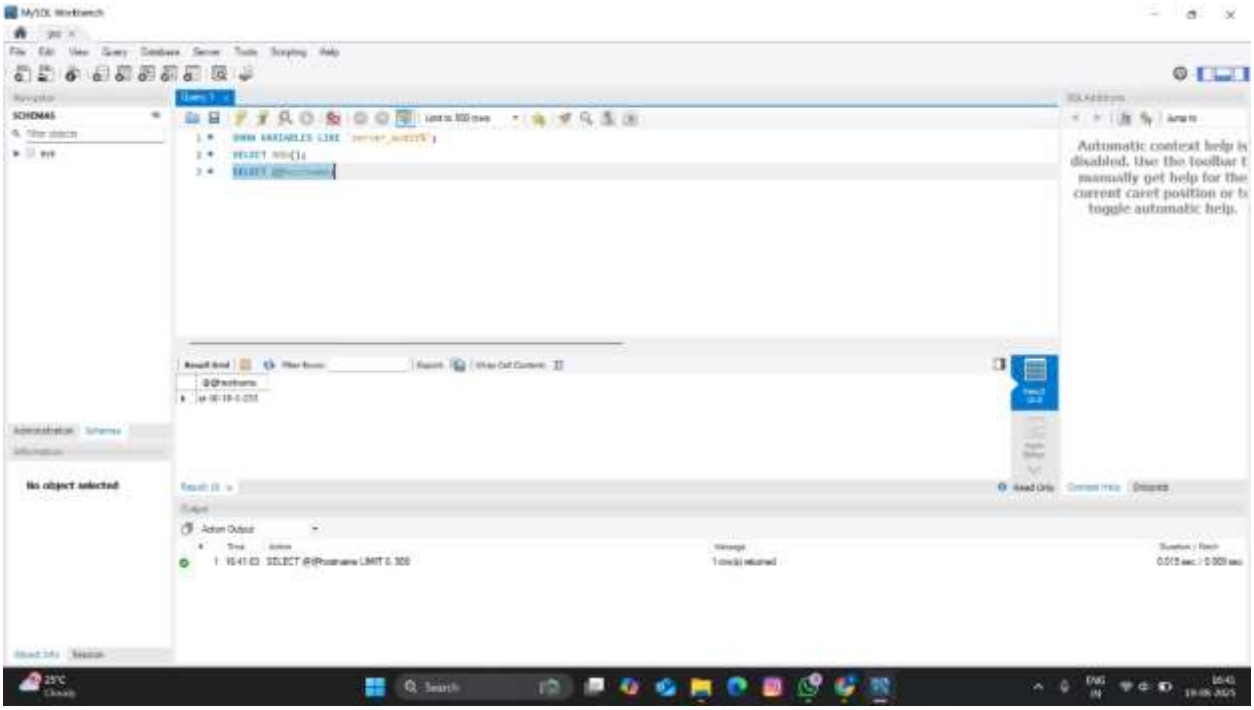
4. Test High Availability (Failover)

1. Connect to DB via endpoint:
2. mysql -h <RDS-endpoint> -u <username> -p
3. Run a sample query:
4. SELECT @@hostname;

(this tells you which underlying DB node you are connected to).

5. **Trigger Failover:**
   - In AWS Console → RDS → Your instance → Actions → Reboot → **Reboot with failover**.
   - The primary will fail over to the standby in the other AZ.
6. **Recheck Connection:**

- Run the query again:
- SELECT @@hostname;
- You'll see a new hostname → confirms **failover occurred**.



Task: Compare Default vs. Custom Parameter Groups

1. Launch Two RDS Instances

- **Instance A → Default Parameter Group**
  - Engine: MySQL (e.g., MySQL 8.0).
  - Leave DB parameter group as **default.mysql8.0**.

- **Instance B → Custom Parameter Group**
  - Create a parameter group custom-mysql-params.
  - Modify settings:
    - time_zone = Asia/Kolkata
    - autocommit = 0 (or 1, depending on your test)
  - Assign this group to **Instance B**.

---

2. Verification Queries

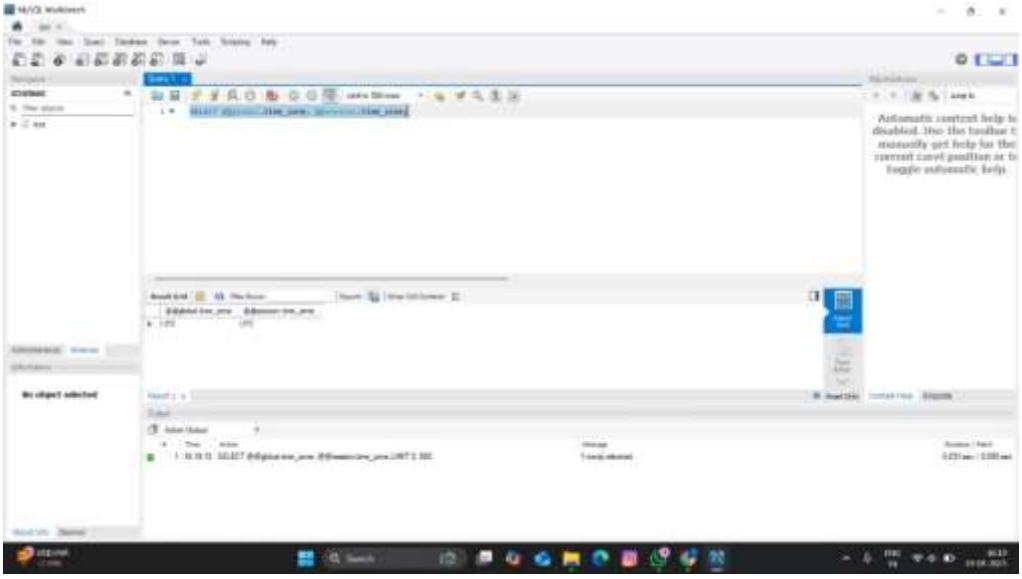
Connect to both instances (using endpoint + mysql client or Workbench).

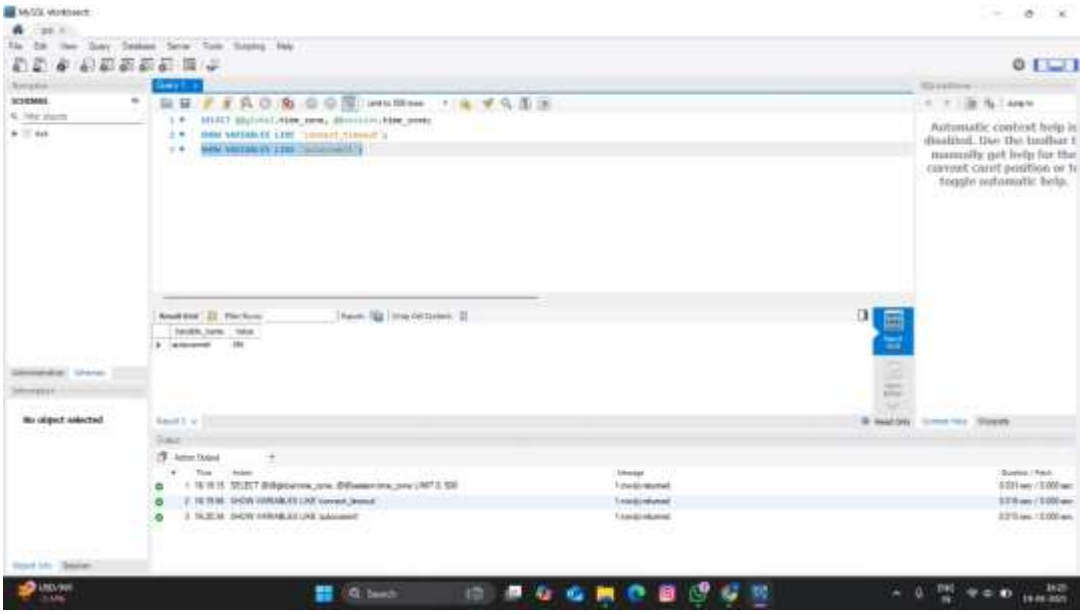
Check Parameters

SHOW VARIABLES LIKE 'time_zone';

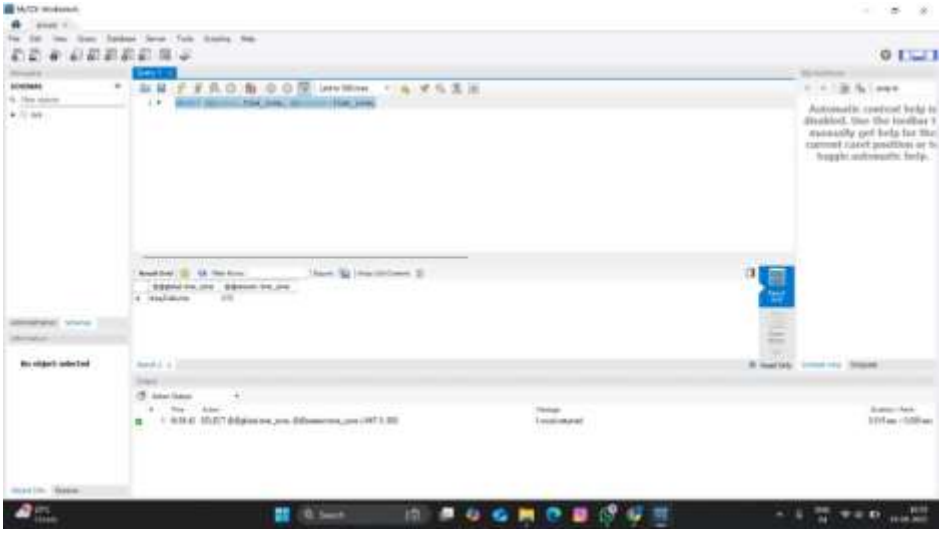
SHOW VARIABLES LIKE 'autocommit';

- Instance A (Default) →
  - Likely time_zone = SYSTEM (usually UTC).
  - autocommit = 1.





- Instance B (Custom) →
    - time_zone = Asia/Kolkata.
    - autocommit = 0.



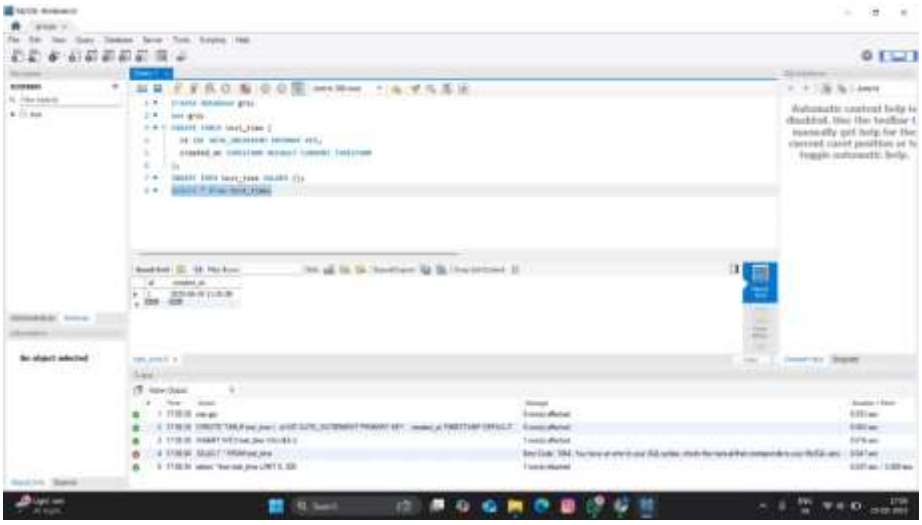
---

3. Behavioral Differences

Test 1 – Timestamp Storage

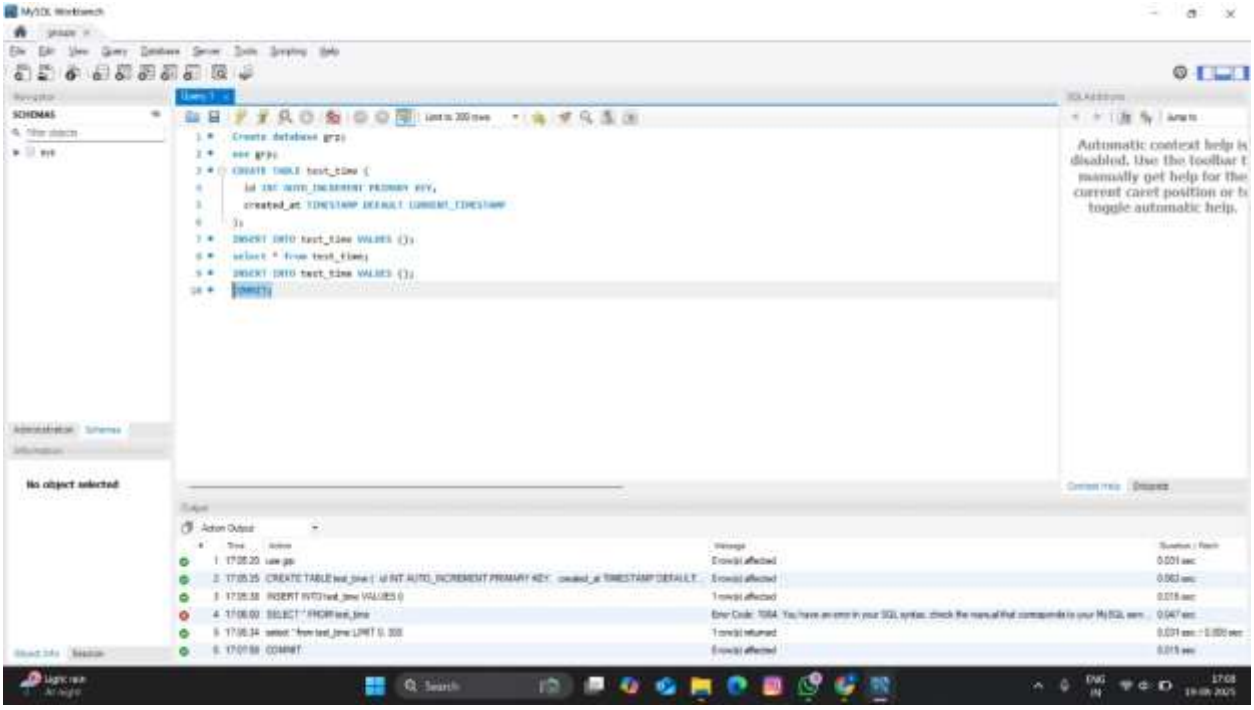
1. On both instances, create a table:
2. CREATE TABLE test_time (
3. id INT AUTO_INCREMENT PRIMARY KEY,
4. created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
5.);
6. Insert a row:
7. INSERT INTO test_time VALUES ();
8. Select the row:
9. SELECT * FROM test_time;

- **Default Group Instance (UTC)** → Timestamps stored in UTC.
- **Custom Group Instance (Asia/Kolkata)** → Timestamps offset by +5:30 hrs.



Test 2 – Autocommit Behavior

1. Run:
2. INSERT INTO test_time VALUES ();
   - In **Default instance (autocommit=1)** → data is immediately committed.
   - In **Custom instance (autocommit=0)** → transaction is open until you explicitly run:
   - COMMIT;



If you close the session without committing, the row won't persist.

Task: Multi-Region Time Zone Simulation

1. Deploy RDS in Two Regions

- **Region A → ap-south-1 (Mumbai)**
- **Region B → us-east-1 (N. Virginia)**

For each:

- Launch an **RDS MySQL** instance.
- Use the same DB engine version (e.g., MySQL 8.0).
- Security group: allow access from your IP.

---

2. Create Region-Specific Parameter Groups

- **Mumbai instance (ap-south-1)**
  - Parameter group: pg-mumbai
  - Set:
  - time_zone = 'Asia/Kolkata'
  - autocommit = 1
- **N. Virginia instance (us-east-1)**
  - Parameter group: pg-virginia
  - Set:
  - time_zone = 'US/Eastern'
  - autocommit = 1

Apply groups → reboot instances.

---

3. Verify Parameters

Connect to each DB:

SHOW VARIABLES LIKE 'time_zone';

Expected:

- Mumbai → Asia/Kolkata
- Virginia → US/Eastern

---

4. Insert Test Data

Create the same table in both regions:

CREATE TABLE user_logins (

```
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Insert identical test rows (same logical action across regions):

```
INSERT INTO user_logins (user_id) VALUES (101);
```

---

5. Compare Results

Query each DB:

```
SELECT * FROM user_logins;
```

- **Mumbai (Asia/Kolkata, UTC+5:30)**
  → 2025-08-19 20:00:00
- **Virginia (US/Eastern, UTC-4:00)**
  → 2025-08-19 10:30:00

Both represent the *same actual moment in time*, but stored/displayed according to **region's local time zone**.

📌 **Real-World Insight**

- If your app users are global, you should **always store timestamps in UTC** (default) and **convert at the application/UI layer**.
- Region-specific parameter groups are useful for **testing and simulation**, but in production, mixing time zones across regions can cause **serious data consistency issues**.

Task: Troubleshooting Parameter Conflicts

1. Intentionally Misconfigure Parameter

- Create or modify a custom parameter group:
  - connect_timeout = 1

    (too aggressive; most clients can't complete SSL handshake or DNS resolve in 1s).
- Apply this group to your RDS MySQL instance.
- **Reboot instance** (since some parameters need reboot).

---

2. Expected Issue

Now try connecting from CLI or Workbench:

mysql -h <rds-endpoint> -u <username> -p

⚠️ Likely outcomes:

- **Connection refused** or **timeout error**:
- ERROR 2003 (HY000): Can't connect to MySQL server on '<endpoint>' (110)
- Some clients may succeed occasionally, but most fail due to the 1s window.

---

3. Troubleshooting & Recovery Steps

1. **Identify Root Cause**
   - Go to **RDS Console → Logs & events → Error log**.
   - Or check client-side error (connect_timeout exceeded).
2. **Fix the Parameter Group**
   - Modify your parameter group:
     - connect_timeout = 60 (default is 10).
   - Save.
3. **Reboot DB Instance**
   - Required for timeout-related changes to take effect.
4. **Validate Fix**
   - Connect again:
   - SHOW VARIABLES LIKE 'connect_timeout';
   - Expected: 60.
   - Retry login — should succeed consistently.

Task: Query Logging with Option Groups (Advanced Security Use Case)

1. Enable General Query Logging via Option Group

1. Go to **AWS Console → RDS → Option groups**.
2. Click **Create group**:
   - Name: query-logging-group
   - Engine: MySQL (match your version).
   - Type: Option group.
3. Add option:
   - Option name: **MARIADB_AUDIT_PLUGIN** (RDS uses MariaDB audit plugin for MySQL).
   - Enable **general query logging** via settings:
     - SERVER_AUDIT_EVENTS = QUERY
     - SERVER_AUDIT_LOGGING = 1
   - Send to **CloudWatch Logs** (create/select log group).

Alternative (parameter group):

- You could also enable the built-in MySQL general log:
- general_log = 1
- log_output = FILE
- But for **security insights**, the audit plugin is recommended since it structures logs better.

---

2. Apply Option Group

- Modify your RDS MySQL instance → under **Option group**, select query-logging-group.
- Save & reboot.

---

3. Run Sample Queries

Connect to RDS and run:

```
-- Normal queries
SELECT NOW();
SELECT * FROM users WHERE user_id = 1;

-- Suspicious (SQL injection style)
SELECT * FROM users WHERE username = 'admin' OR '1'='1';
```

SELECT * FROM users WHERE id = 10; DROP TABLE users;

---

4. View & Analyze Logs

1. **RDS Console** → DB Instance → Logs & Events → audit log.
2. **CloudWatch** → Log Groups → chosen audit log group.

You'll see entries like:

[Server_audit] Query; user=app_user; SELECT NOW();

[Server_audit] Query; user=app_user; SELECT * FROM users WHERE username = 'admin' OR '1'='1';

---

5. Security Insights

- **Normal usage**: straightforward SELECT, INSERT, UPDATE.
- **Injection detection**: suspicious patterns like:
  - ' OR '1'='1 (classic tautology injection).
  - ; DROP TABLE (stacked query attempt).
  - Excessive failed logins (from CONNECT events).

By monitoring these logs, you can:

- Detect anomalous queries in real-time.
- Create **CloudWatch metric filters** for suspicious patterns.
- Trigger **SNS/CloudWatch Alarms** for alerting.